

Statistical Computing

Monte Carlo Integration and Variance Reduction

Spring 2026

Outline

Monte Carlo Integration

Variance Reduction

Antithetic Variables

Control Variates

Importance Sampling

Stratified Sampling

Monte Carlo Integration

Expectation identity

Let $g(x)$ be a function and suppose that we want to compute

$$\int_a^b g(x) dx \quad (\text{assuming that this integral exists}). \quad (1)$$

If X is a random variable with density $f(x)$ and $Y = g(X)$, then

$$\mathbb{E}[g(X)] = \int_{-\infty}^{\infty} g(x) f(x) dx. \quad (2)$$

If a random sample is available from the distribution of X , an unbiased estimator of $\mathbb{E}[g(X)]$ is the sample mean.

Simple Monte Carlo Estimator

Setup

Consider estimating

$$\theta = \int_0^1 g(x) dx. \quad (3)$$

If $X_1, \dots, X_m$ is a random $\text{Uniform}(0, 1)$ sample, then

$$\hat{\theta} = \bar{g}_m(X) = \frac{1}{m} \sum_{i=1}^m g(X_i) \quad (4)$$

converges to $\mathbb{E}[g(X)] = \theta$ with probability 1 (Strong Law of Large Numbers).

Python: Simple Monte Carlo integration

Code

```
import numpy as np
m = 10000
seed = 2026
rng = np.random.default_rng(seed)
x = rng.random(m) # Uniform(0,1)
theta_hat = np.mean(np.exp(-x)) #  $g(x)=\exp(-x)$ 
print(theta_hat)
print(1 - np.exp(-1))
```

Change of limits

From (a, b) to $(0, 1)$

To compute $\int_a^b g(t) dt$, use the linear transformation

$$y = \frac{t - a}{b - a}, \quad dy = \frac{1}{b - a} dt. \quad (5)$$

Then

$$\int_a^b g(t) dt = \int_0^1 g(y(b - a) + a) (b - a) dy. \quad (6)$$

Uniform(a, b) sampling

Equivalently, if $Y \sim \text{Uniform}(a, b)$, then

$$\int_a^b g(t) dt = (b - a) \mathbb{E}[g(Y)]. \quad (7)$$

Python: Simple Monte Carlo integration on $(2, 4)$

Code

```
m = 10000
x = rng.uniform(2.0, 4.0, size=m) # Uniform(2,4)
theta_hat = np.mean(np.exp(-x)) * 2.0 # (b-a)=2
print(theta_hat)
print(np.exp(-2) - np.exp(-4))
```

Simple Monte Carlo estimator for $\int_a^b g(x) dx$

Algorithm

1. Generate $X_1, \dots, X_m$ iid from $\text{Uniform}(a, b)$.
2. Compute $\bar{g}(X) = \frac{1}{m} \sum_{i=1}^m g(X_i)$.
3. Set $\hat{\theta} = (b - a) \bar{g}(X)$.

Monte Carlo integration: unbounded interval

Standard normal cdf

Estimate

$$\Phi(x) = \int_{-\infty}^x \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt. \quad (8)$$

For $x > 0$, write

$$\theta = \int_0^x e^{-t^2/2} dt. \quad (9)$$

Using $y = t/x$ (so $dt = x dy$),

$$\theta = \int_0^1 x e^{-(xy)^2/2} dy = \mathbb{E} \left[x e^{-(xY)^2/2} \right], \quad Y \sim \text{Uniform}(0, 1). \quad (10)$$

- If $x > 0$, estimate $\Phi(x)$ by $0.5 + \hat{\theta} / \sqrt{2\pi}$.
- If $x < 0$, use $\Phi(x) = 1 - \Phi(-x)$.

Python: Monte Carlo estimate of $\Phi(x)$ via $\text{Uniform}(0,1)$

Code

```
from scipy.stats import norm
x = np.linspace(0.1, 2.5, 10)
m = 10000
u = rng.random(m) # Uniform(0,1)
cdf = np.empty_like(x)
for i, xi in enumerate(x):
    g = xi * np.exp(-(u * xi) ** 2 / 2.0)
    cdf[i] = np.mean(g) / np.sqrt(2 * np.pi) + 0.5
Phi = norm.cdf(x)
print(np.round(np.vstack([x, cdf, Phi]), 3))
```

Indicator-function estimator of $\Phi(x)$

Setup

Let $I(\cdot)$ be the indicator function and $Z \sim N(0, 1)$. Then

$$\mathbb{E}[I(Z \leq x)] = \mathbb{P}(Z \leq x) = \Phi(x). \quad (11)$$

Generate $z_1, \dots, z_m$ from the standard normal distribution. Then

$$\hat{\Phi}(x) = \frac{1}{m} \sum_{i=1}^m I(z_i \leq x) \quad (12)$$

converges with probability one to $\Phi(x)$.

Python: Indicator estimator of $\Phi(x)$

Code

```
x = np.linspace(0.1, 2.5, 10)
m = 10000
z = rng.standard_normal(m)
p = (z[:, None] < x[None, :]).mean(axis=0)
Phi = norm.cdf(x)
print(np.round(np.vstack([x, p, Phi]), 3))
```

Variance estimate

Compute the sample mean

$$\hat{\theta} = \frac{1}{m} \sum_{i=1}^m g(x_i). \quad (13)$$

When the distribution of X is unknown, substitute the empirical distribution F_m of $x_1, \dots, x_m$. The variance of $\hat{\theta}$ can be estimated by

$$\hat{\sigma}_m^2 = \frac{1}{m^2} \sum_{i=1}^m [g(x_i) - \bar{g}]^2, \quad \bar{g} = \frac{1}{m} \sum_{i=1}^m g(x_i). \quad (14)$$

Standard error and CLT

The corresponding estimate of standard error is

$$\widehat{se}(\hat{\theta}) = \frac{1}{m} \left[\sum_{i=1}^m (g(x_i) - \bar{g})^2 \right]^{1/2}. \quad (15)$$

The Central Limit Theorem implies that

$$\frac{\hat{\theta} - \mathbb{E}[\hat{\theta}]}{\sqrt{\text{Var}(\hat{\theta})}} \Rightarrow N(0, 1) \quad (m \rightarrow \infty). \quad (16)$$

Python: Approx. 95% CI for $\Phi(2)$ and $\Phi(2.5)$

Code

```
m = 10000
z = rng.standard_normal(m)
for x in [2.0, 2.5]:
    g = (z < x).astype(float) # indicator
    cdf = np.mean(g)
    v = np.mean((g - cdf) ** 2) / m # variance estimate
    print(cdf, v)
    print(cdf - 1.96 * np.sqrt(v), cdf + 1.96 * np.sqrt(v))
```

Variance of the sample-mean estimator

If $X \sim \text{Uniform}(a, b)$, then

$$\theta = \int_a^b g(x) dx = (b - a) \mathbb{E}[g(X)]. \quad (17)$$

With $\hat{\theta} = (b - a) \bar{g}(X)$ and $\bar{g}(X) = \frac{1}{m} \sum_{i=1}^m g(X_i)$,

$$\mathbb{E}[\hat{\theta}] = \theta, \quad \text{Var}(\hat{\theta}) = \frac{(b - a)^2}{m} \text{Var}(g(X)). \quad (18)$$

By the Central Limit Theorem, for large m , $\hat{\theta}$ is approximately normal with mean θ and the variance above.

Hit-or-miss approach for $F(x)$

Procedure

Suppose $f(x)$ is the density of a random variable X . To estimate

$$F(x) = \int_{-\infty}^x f(t) dt, \quad (19)$$

1. Generate $X_1, \dots, X_m$ from the distribution of X .
2. Compute $g(X_i) = I(X_i \leq x)$ for each i .
3. Compute $\hat{F}(x) = \frac{1}{m} \sum_{i=1}^m I(X_i \leq x)$.

Hit-or-miss approach for $F(x)$

Variance

Here $Y = g(X)$ is Bernoulli(p) with $p = F(x)$, so

$$\text{Var}(\hat{F}(x)) = \frac{F(x)(1 - F(x))}{m}. \quad (20)$$

It can be estimated by $\hat{F}(x)(1 - \hat{F}(x))/m$. The maximum variance occurs when $F(x) = 1/2$, giving the conservative bound $1/(4m)$.

Definition

If $\hat{\theta}_1$ and $\hat{\theta}_2$ are two estimators for θ , then $\hat{\theta}_1$ is more efficient (in a statistical sense) than $\hat{\theta}_2$ if

$$\frac{\text{Var}(\hat{\theta}_1)}{\text{Var}(\hat{\theta}_2)} < 1. \quad (21)$$

- If the variances are unknown, estimate efficiency by substituting a sample estimate of the variance for each estimator.
- Variance can always be reduced by increasing the number of replicates, so computational efficiency is also relevant.

Variance Reduction

- Here we consider approaches to reducing the variance in the sample mean estimator of $\theta = \mathbb{E}[g(X)]$.

Replicates

The Monte Carlo approach to estimating $\theta = \mathbb{E}[g(X)]$ is to compute the sample mean of $g(X)$ for a large number m of replicates from the distribution of $g(X)$. The function $g(\cdot)$ is often a statistic, an n -variate function $g(X_1, \dots, X_n)$.

Unbiasedness and variance

Let

$$X^{(j)} = \{X_1^{(j)}, \dots, X_n^{(j)}\}, \quad j = 1, \dots, m \quad (22)$$

be iid from the distribution of X , and define

$$Y_j = g(X_1^{(j)}, \dots, X_n^{(j)}), \quad j = 1, \dots, m. \quad (23)$$

Unbiasedness and variance

Then $Y_1, \dots, Y_m$ are iid with distribution of $Y = g(X)$, and

$$\mathbb{E} \left[\frac{1}{m} \sum_{j=1}^m Y_j \right] = \theta. \quad (24)$$

Thus $\hat{\theta} = \bar{Y}$ is unbiased for $\theta = \mathbb{E}[Y]$, with

$$\text{Var}(\hat{\theta}) = \text{Var}(\bar{Y}) = \frac{\text{Var}(g(X))}{m}. \quad (25)$$

Replicates required and motivation

- Increasing the number of replicates m reduces the variance of the Monte Carlo estimator.
- However, a large increase in m is needed to get even a small improvement in standard error.
- Rule of thumb: If the standard error should be at most e and $\text{Var}(g(X)) = \sigma^2$, then approximately

$$m \gtrsim \sigma^2 / e^2 \quad (26)$$

replicates are required.

- Other variance-reduction methods can be less computationally expensive than simply increasing m .

Antithetic Variables

Variance of an average

Consider the mean of two identically distributed random variables U_1 and U_2 . If U_1 and U_2 are independent, then

$$\text{Var}\left(\frac{U_1 + U_2}{2}\right) = \frac{1}{4}(\text{Var}(U_1) + \text{Var}(U_2)), \quad (27)$$

but in general,

$$\text{Var}\left(\frac{U_1 + U_2}{2}\right) = \frac{1}{4}(\text{Var}(U_1) + \text{Var}(U_2) + 2\text{Cov}(U_1, U_2)). \quad (28)$$

Thus, $\text{Var}((U_1 + U_2)/2)$ is smaller if U_1 and U_2 are negatively correlated than when they are independent. This motivates using negatively correlated variables to reduce variance.

Inverse transform construction

Suppose that $X_1, \dots, X_n$ are simulated via the inverse transform method. For each replicate, generate $U \sim \text{Uniform}(0, 1)$ and set

$$X = F_X^{-1}(U). \quad (29)$$

Note that if $U \sim \text{Uniform}(0, 1)$ then $1 - U$ has the same distribution as U , but U and $1 - U$ are negatively correlated.

Antithetic replicate

In the Monte Carlo replicates,

$$Y_j = g\left(F_X^{-1}(U_1^{(j)}), \dots, F_X^{-1}(U_n^{(j)})\right) \quad (30)$$

has the same distribution as

$$Y'_j = g\left(F_X^{-1}(1 - U_1^{(j)}), \dots, F_X^{-1}(1 - U_n^{(j)})\right). \quad (31)$$

Under what conditions are Y_j and Y'_j negatively correlated?

Partial order and monotonicity

Define $(x_1, \dots, x_n) \preceq (y_1, \dots, y_n)$ if $x_j \leq y_j$ for $j = 1, \dots, n$. An n -variate function $g = g(X_1, \dots, X_n)$ is increasing if

$$g(x_1, \dots, x_n) \leq g(y_1, \dots, y_n) \quad \text{whenever} \quad (x_1, \dots, x_n) \preceq (y_1, \dots, y_n). \quad (32)$$

Similarly, g is decreasing if it is decreasing in its coordinates. Then g is monotone if it is increasing or decreasing.

Proposition

If $X_1, \dots, X_n$ are independent, and f and g are increasing functions, then

$$\mathbb{E}[f(X)g(X)] \geq \mathbb{E}[f(X)] \mathbb{E}[g(X)]. \quad (33)$$

Corollary

If $g = g(X_1, \dots, X_n)$ is monotone, then

$$Y = g\left(F_X^{-1}(U_1), \dots, F_X^{-1}(U_n)\right) \quad \text{and} \quad Y' = g\left(F_X^{-1}(1 - U_1), \dots, F_X^{-1}(1 - U_n)\right) \quad (34)$$

are negatively correlated.

Proof (sketch)

Without loss of generality suppose that g is increasing. Then Y and $-Y'$ are both increasing functions, so $\mathbb{E}[Y(-Y')] \geq \mathbb{E}[Y]\mathbb{E}[-Y']$, i.e., $\mathbb{E}[YY'] \leq \mathbb{E}[Y]\mathbb{E}[Y']$. Thus $\text{Cov}(Y, Y') \leq 0$ and Y, Y' are negatively correlated.

Construction

If m Monte Carlo replicates are required, generate $m/2$ replicates

$$Y_j = g\left(F_X^{-1}(U_1^{(j)}), \dots, F_X^{-1}(U_n^{(j)})\right), \quad j = 1, \dots, m/2, \quad (35)$$

and the remaining $m/2$ replicates

$$Y'_j = g\left(F_X^{-1}(1 - U_1^{(j)}), \dots, F_X^{-1}(1 - U_n^{(j)})\right), \quad j = 1, \dots, m/2, \quad (36)$$

where $U_i^{(j)}$ are iid Uniform(0, 1).

Estimator

Then the antithetic estimator is

$$\hat{\theta} = \frac{1}{m} \left(Y_1 + Y'_1 + \cdots + Y_{m/2} + Y'_{m/2} \right) = \frac{2}{m} \sum_{j=1}^{m/2} \left(\frac{Y_j + Y'_j}{2} \right). \quad (37)$$

Thus $nm/2$ rather than nm uniform variates are required, and the variance is reduced by using antithetic variables.

Example: Antithetic variables for $\Phi(x)$

Setup

Refer to the Monte Carlo integration example for estimating the standard normal cdf

$$\Phi(x) = \int_{-\infty}^x \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt. \quad (38)$$

Repeat the estimation using antithetic variables and find the approximate reduction in standard error. After change of variables, the target parameter is

$$\theta = \mathbb{E}_U \left[x e^{-(xU)^2/2} \right], \quad U \sim \text{Uniform}(0, 1). \quad (39)$$

By restricting to the case $x > 0$, the function

$$g(u) = x e^{-(xu)^2/2}$$

is monotone decreasing in u , so the hypothesis of the corollary is satisfied.

Example: antithetic construction

Replicates

Generate $u_1, \dots, u_{m/2} \sim \text{Uniform}(0, 1)$ and compute

$$Y_j = x e^{-(u_j x)^2/2}, \quad j = 1, \dots, m/2, \quad (40)$$

and the remaining half using

$$Y'_j = x e^{-((1-u_j)x)^2/2}, \quad j = 1, \dots, m/2. \quad (41)$$

Example: antithetic construction

Sample mean

The sample mean

$$\begin{aligned}\hat{\theta} = g_m(u) &= \frac{1}{m} \sum_{j=1}^{m/2} \left(x e^{-(u_j x)^2/2} + x e^{-((1-u_j)x)^2/2} \right) \\ &= \frac{1}{m/2} \sum_{j=1}^{m/2} \left[\frac{x e^{-(u_j x)^2/2} + x e^{-((1-u_j)x)^2/2}}{2} \right]\end{aligned}\tag{42}$$

converges to $\mathbb{E}[\hat{\theta}] = \theta$ as $m \rightarrow \infty$. If $x > 0$, the estimate of $\Phi(x)$ is $0.5 + \hat{\theta}/\sqrt{2\pi}$. If $x < 0$, compute $\Phi(x) = 1 - \Phi(-x)$.

Python: Monte Carlo estimation of $\Phi(x)$ (optional antithetic)

Code

```
def mc_phi(x, R=10000, antithetic=True, rng=None, seed=2026):
    if rng is None:
        rng = np.random.default_rng(seed)
    u = rng.random(R // 2)
    v = (1.0 - u) if antithetic else rng.random(R // 2)
    u = np.concatenate([u, v])
    x = np.atleast_1d(x).astype(float)
    cdf = np.empty_like(x)
    for i, xi in enumerate(x):
        g = xi * np.exp(-(u * xi) ** 2 / 2.0)
        cdf[i] = np.mean(g) / np.sqrt(2.0 * np.pi) + 0.5
    return cdf
```

Python: Single Monte Carlo experiment (comparison table)

Code

```
x = np.linspace(0.1, 2.5, 5)
Phi = norm.cdf(x)
MC1 = mc_phi(x, R=10000, antithetic=False, seed=123)
MC2 = mc_phi(x, R=10000, antithetic=True, seed=123)
print(np.round(np.vstack([x, MC1, MC2, Phi]), 5))
```

Python: Estimate reduction in variance by simulation

Code

```
m=1000
rng = np.random.default_rng(123)
MC1 = np.zeros(m)
MC2 = np.zeros(m)
for i in range(m):
    MC1[i] = mc_phi(1.95, R=1000, antithetic=False, rng=rng)[0]
    MC2[i] = mc_phi(1.95, R=1000, antithetic=True, rng=rng)[0]
print(np.std(MC1))
print(np.std(MC2))
print((np.var(MC1) - np.var(MC2)) / np.var(MC1))
```

Control Variates

Control Variates

Setup

Another approach to reduce the variance in a Monte Carlo estimator of

$$\theta = \mathbb{E}[g(X)] \quad (43)$$

is the use of control variates. Suppose that there is a function f such that

$$\mu = \mathbb{E}[f(X)] \quad (44)$$

is known, and $f(X)$ is correlated with $g(X)$.

Controlled estimator

For any constant c ,

$$\hat{\theta}_c = g(X) + c(f(X) - \mu) \quad (45)$$

is an unbiased estimator of θ .

Variance and optimal constant

The variance is

$$\text{Var}(\hat{\theta}_c) = \text{Var}(g(X)) + c^2 \text{Var}(f(X)) + 2c \text{Cov}(g(X), f(X)), \quad (46)$$

a quadratic function of c , minimized at

$$c^* = -\frac{\text{Cov}(g(X), f(X))}{\text{Var}(f(X))}. \quad (47)$$

Minimum variance

At $c = c^*$,

$$\text{Var}(\hat{\theta}_{c^*}) = \text{Var}(g(X)) - \frac{[\text{Cov}(g(X), f(X))]^2}{\text{Var}(f(X))}. \quad (48)$$

Percent reduction in variance

In the minimum-variance expression above, $\text{Var}(g(X))$ is reduced by

$$\frac{[\text{Cov}(g(X), f(X))]^2}{\text{Var}(f(X))}. \quad (49)$$

Hence the percent reduction in variance is

$$100 \frac{[\text{Cov}(g(X), f(X))]^2}{\text{Var}(g(X)) \text{Var}(f(X))} = 100 [\text{Cor}(g(X), f(X))]^2. \quad (50)$$

- It is advantageous if $f(X)$ and $g(X)$ are strongly correlated.
- No reduction of variance is possible if $f(X)$ and $g(X)$ are uncorrelated.
- The parameters needed for c^* can be estimated from a preliminary Monte Carlo experiment.

Example: Control variate for $\mathbb{E}[e^U]$

Target and simple Monte Carlo variance

Apply the control variate approach to compute

$$\theta = \mathbb{E}[e^U] = \int_0^1 e^u du, \quad U \sim \text{Uniform}(0, 1). \quad (51)$$

Here $\theta = e - 1 = 1.718282$. If the simple Monte Carlo approach is applied with m replicates, the variance of the estimator is $\text{Var}(g(U))/m$, where

$$\text{Var}(g(U)) = \text{Var}(e^U) = \mathbb{E}[e^{2U}] - \theta^2 = \frac{e^2 - 1}{2} - (e - 1)^2 = 0.2420351. \quad (52)$$

Example: Control variate for $\mathbb{E}[e^U]$

Control variate and optimal constant

A natural choice for a control variate is $U \sim \text{Uniform}(0, 1)$. Then $\mathbb{E}[U] = 1/2$, $\text{Var}(U) = 1/12$, and

$$\text{Cov}(e^U, U) = 1 - \frac{1}{2}(e - 1) = 0.1408591. \quad (53)$$

Hence

$$c^* = -\frac{\text{Cov}(e^U, U)}{\text{Var}(U)} = -12 + 6(e - 1) = -1.690309. \quad (54)$$

Our controlled estimator is

$$\hat{\theta}_{c^*} = e^U - 1.690309 (U - 0.5). \quad (55)$$

Example: Control variate for $\mathbb{E}[e^U]$

Minimum variance and percent reduction

For m replicates, $m \text{Var}(\hat{\theta}_{c^*})$ is

$$\text{Var}(e^U) - \frac{[\text{Cov}(e^U, U)]^2}{\text{Var}(U)} = 0.003940175. \quad (56)$$

The percent reduction in variance is approximately 98.3781%.

Python: Control variate for $\mathbb{E}[e^U]$

Code

```
import numpy as np
m = 10000
rng = np.random.default_rng(1)
a = -12.0 + 6.0 * (np.e - 1.0)
U = rng.random(m)
T1 = np.exp(U) # simple MC
T2 = np.exp(U) + a * (U - 0.5) # controlled
print(np.mean(T1))
print(np.mean(T2))
print((np.var(T1, ddof=1) - np.var(T2, ddof=1)) / np.var(T1, ddof=1))
```

Importance Sampling

Average value on an interval

The average value of a function $g(x)$ over an interval (a, b) is

$$\frac{1}{b-a} \int_a^b g(x) dx. \quad (57)$$

If $X \sim \text{Uniform}(a, b)$, then

$$\mathbb{E}[g(X)] = \int_a^b g(x) \frac{1}{b-a} dx = \frac{1}{b-a} \int_a^b g(x) dx, \quad (58)$$

which is the average value of $g(x)$ with respect to a uniform weight function.

Simple Monte Carlo

Generate $X_1, \dots, X_m \sim \text{Uniform}(a, b)$ and estimate $\int_a^b g(x) dx$ by

$$(b - a) \frac{1}{m} \sum_{i=1}^m g(X_i), \quad (59)$$

which converges with probability 1 by the strong law of large numbers.

- A limitation: it does not apply to unbounded intervals.
- Another drawback: it can be inefficient if $g(x)$ is not very uniform over (a, b) .

General weight function

Consider using a density $f(x)$ (not necessarily uniform). Assume that

$$f(x) > 0 \quad \text{on the set } \{x : g(x) > 0\}. \quad (60)$$

Let

$$Y = \frac{g(X)}{f(X)}, \quad X \sim f. \quad (61)$$

Then

$$\int g(x) dx = \int \frac{g(x)}{f(x)} f(x) dx = \mathbb{E}[Y]. \quad (62)$$

Importance sampling estimator

Generate $X_1, \dots, X_m \sim f$ and compute

$$\hat{\theta} = \frac{1}{m} \sum_{i=1}^m \frac{g(X_i)}{f(X_i)}. \quad (63)$$

The density $f(x)$ is called the *importance function*.

- The variance is $\text{Var}(Y)/m$; variance is small if Y is nearly constant.
- Hence $f(\cdot)$ should be “close” to $g(\cdot)$ and reasonably easy to simulate.

Importance Sampling

Envelope form on a set A

Suppose that $f(x)$ is a density supported on a set A . If $\phi(x) > 0$ on A , then

$$\theta = \int_A g(x)f(x) dx = \int_A \frac{g(x)f(x)}{\phi(x)} \phi(x) dx. \quad (64)$$

If $\phi(\cdot)$ is a density on A , then an estimator of $\theta = \mathbb{E}_\phi[g(X)f(X)/\phi(X)]$ is

$$\hat{\theta} = \frac{1}{n} \sum_{i=1}^n \frac{g(X_i)f(X_i)}{\phi(X_i)}, \quad X_1, \dots, X_n \sim \phi. \quad (65)$$

The function $\phi(\cdot)$ is called the *envelope* or the *importance sampling function*.

- Typically choose $\phi(x)$ so that $\phi(x) \propto |g(x)|f(x)$ on A (and ϕ has finite variance).
- If $g(x)/\phi(x)$ is bounded, then the importance sampling estimator has finite variance.

Example: Choice of the importance function

Target integral

Compare several importance functions to estimate

$$\int_0^1 \frac{e^{-x}}{1+x^2} dx. \quad (66)$$

Define the integrand on $\mathbb{R}$ by

$$g(x) = \begin{cases} \frac{e^{-x}}{1+x^2}, & 0 < x < 1, \\ 0, & \text{otherwise.} \end{cases} \quad (67)$$

Example: Choice of the importance function

Candidate importance functions

$$f_0(x) = 1, \quad 0 < x < 1, \quad (68)$$

$$f_1(x) = e^{-x}, \quad 0 < x < \infty, \quad (69)$$

$$f_2(x) = \frac{1}{\pi(1+x^2)}, \quad -\infty < x < \infty, \quad (70)$$

$$f_3(x) = \frac{e^{-x}}{1-e^{-1}}, \quad 0 < x < 1, \quad (71)$$

$$f_4(x) = \frac{4}{\pi(1+x^2)}, \quad 0 < x < 1. \quad (72)$$

- Although all are positive on $(0, 1)$, f_1 and f_2 generate many samples outside $(0, 1)$, producing zeros in $g(x)/f(x)$ (inefficient).

Python: Importance sampling comparison (five choices)

Code

```
m = 10000
theta_hat = np.zeros(5)
sd_hat = np.zeros(5)
def g(x):
    return np.exp(-x) / (1.0 + x ** 2) * (x > 0.0) * (x < 1.0)
# f0: Uniform(0,1)
x = rng.random(m)
fg = g(x) # divide by f0(x)=1 on (0,1)
theta_hat[0], sd_hat[0] = fg.mean(), fg.std(ddof=1)
# f1: Exp(rate=1), support (0, inf)
x = rng.exponential(scale=1.0, size=m)
fg = g(x) / np.exp(-x) # divide by f1(x)=exp(-x)
theta_hat[1], sd_hat[1] = fg.mean(), fg.std(ddof=1)
```

Python: Importance sampling comparison (f_2, f_3, f_4)

Code

```
x = rng.standard_cauchy(m)
x[(x < 0) | (x > 1)] = 2. # to catch overflow errors in g(x)
fg = g(x) / (1.0 / (np.pi * (1.0 + x**2)))
theta_hat[2], sd_hat[2] = fg.mean(), fg.std(ddof=1)
u = rng.random(m)
x = -np.log(1.0 - u * (1.0 - np.exp(-1.0)))
fg = g(x) / (np.exp(-x) / (1.0 - np.exp(-1.0)))
theta_hat[3], sd_hat[3] = fg.mean(), fg.std(ddof=1)
u = rng.random(m)
x = np.tan(np.pi * u / 4.0)
fg = g(x) / (4.0 / (np.pi * (1.0 + x**2)))
theta_hat[4], sd_hat[4] = fg.mean(), fg.std(ddof=1)
print(np.vstack([theta_hat, sd_hat / np.sqrt(m)]))
```

Example output (as shown in the text)

Estimates and standard errors

	0	1	2	3	4
θ_{hat}	0.524114007	0.531358351	0.5461507	0.5250698759	0.526049238
$\hat{se}$	0.002436559	0.004181264	0.0096613	0.0009658794	0.001427685

- The simulation indicates that f_3 (and possibly f_4) produces the smallest variance among these five.
- The standard Monte Carlo estimate without importance sampling has $\hat{se} \approx 0.00244$ (using $f_0 = 1$).
- The importance functions f_1 and f_2 do not reduce error, but f_3 and f_4 each reduce the standard error.

Why f_1 and f_2 can be inefficient

Many zeros and large variance

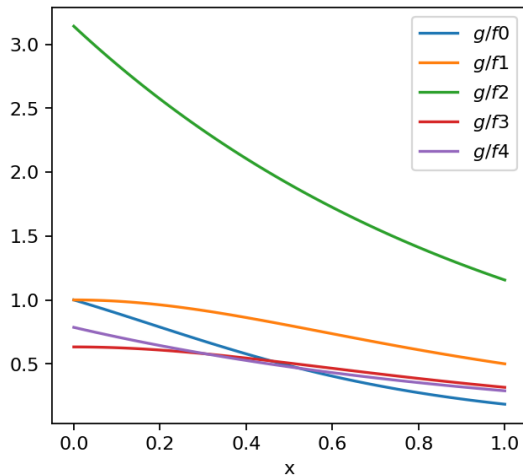
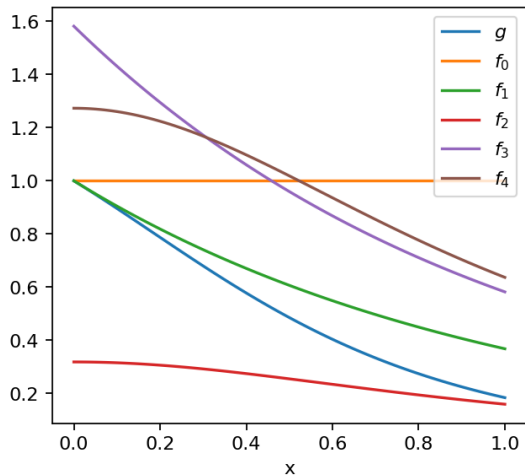
- The Cauchy density f_2 is supported on the entire real line, while $g(x)$ is evaluated on $(0, 1)$.
- A very large number of zeros (about 75%) are produced in the ratio $g(x)/f_2(x)$, and the nonzero values are far from 0, resulting in a large variance.
- For f_1 there is a similar inefficiency, since f_1 is supported on $(0, \infty)$, generating many zeros in $g(x)/f_1(x)$ for values outside $(0, 1)$.

Python: Plot importance functions vs. $g(x)$ on $(0, 1)$

Code

```
import matplotlib.pyplot as plt
x = np.linspace(0, 1, 100)
g = np.exp(-x) / (1.0 + x**2)
f0 = np.ones_like(x); f1 = np.exp(-x); f2 = 1.0 / (np.pi * (1.0 + x**2))
f3 = np.exp(-x) / (1.0 - np.exp(-1.0)); f4 = 4.0 / (np.pi * (1.0 + x**2))
fig, ax = plt.subplots(1,2, figsize=(10,4))
ax[0].plot(x, g, label="$g$")
for i, f in enumerate([f0, f1, f2, f3, f4]):
    ax[0].plot(x, f, label="$f_{%d}$"%i)
    ax[1].plot(x, g/f, label="$g/f_{%d}$"%i)
ax[0].legend(); ax[1].legend()
ax[0].set_xlabel("x"); ax[1].set_xlabel("x")
plt.show()
```

Figure: Importance functions and the integrand



Stratified Sampling

- Another approach to variance reduction is stratified sampling.
- The idea is to divide the interval into strata and estimate the integral on each stratum with smaller variance.
- With total replicates m and stratum sizes $m_1, \dots, m_k$ (with $m = m_1 + \dots + m_k$), the goal is

$$\text{Var}(\hat{\theta}_k(m_1, \dots, m_k)) < \text{Var}(\hat{\theta}),$$

where $\hat{\theta}_k$ is the stratified estimator and $\hat{\theta}$ is the standard Monte Carlo estimator based on m replicates.

Example: Four strata

Setup

From the earlier plot, the integrand $g(x)$ is not constant on $(0, 1)$. Divide $(0, 1)$ into four subintervals, use $1/4$ of the total replicates on each stratum, and combine the four estimates to estimate

$$\int_0^1 \frac{e^{-x}}{1+x^2} dx.$$

Does it appear that the variance is reduced compared with the standard Monte Carlo estimator?

Python: Stratified sampling (four strata)

Code

```
M = 20 # total replicates
estimates = np.zeros((10, 2))
def g(x):
    return np.exp(-x) / (1.0 + x ** 2) * (x > 0.0) * (x < 1.0)
for i in range(10):
    estimates[i, 0] = np.mean(g(rng.random(M)))
    # 4 equal strata, M/4 replicates per stratum
    T2 = np.empty(4)
    T2[0] = np.mean(g(rng.uniform(0.00, 0.25, size=M//4)))
    T2[1] = np.mean(g(rng.uniform(0.25, 0.50, size=M//4)))
    T2[2] = np.mean(g(rng.uniform(0.50, 0.75, size=M//4)))
    T2[3] = np.mean(g(rng.uniform(0.75, 1.00, size=M//4)))
    estimates[i, 1] = np.mean(T2)
print(estimates.mean(axis=0), estimates.var(axis=0, ddof=1))
```

When stratification helps

- Intuitively, stratification can yield more reduction in variance when the means of the strata are widely dispersed.
- If the means of the strata are approximately equal, the reduction can be small.
- For monotone integrands, stratification similar to the example above should be effective.

Proposition

Denote the standard Monte Carlo estimator with M replicates by $\hat{\theta}_M$, and let

$$\hat{\theta}_S = \frac{1}{k} \sum_{j=1}^k \hat{\theta}_j$$

denote the stratified estimator with equal size $m = M/k$ strata. Let the mean and variance of $g(U)$ on stratum j be θ_j and σ_j^2 , respectively. Then

$$\text{Var}(\hat{\theta}_M) \geq \text{Var}(\hat{\theta}_S).$$